

← Main menu

Integrate Generative AI Into Your Apps with Firebase Genkit

Course • 6 hours80% complete

Course overview

Integrate Generative AI Into Your Apps with Firebase Genkit

✓ Getting Started with Firebase Genkit

✓ Function Calling with Firebase Genkit

✓ Deploy Firebase Genkit Apps to Google Cloud

✓ Build a RAG Solution with Firebase Genkit

Build Generative AI Applications with Firebase Genkit: Challenge Lab

Your Next Steps

→

Course > Integrate Generative AI Into Your Apps with Firebase Genkit >

Lab setup instructions and requirements

Protect your account and progress. Always use a private browser window and lab credentials to run this lab.

View full setup instructions

End Lab

00:21:12

Caution: When you are in the console, do not deviate from the lab instructions. Doing so may cause your account to be blocked. Learn more.

Open Google Cloud Console

Username

student-82-3fd3c3684cfdi

Password

AxH1S2QXXQy0

GCP Project ID

qwiklabs-gcp-83-9ec807d1

Build Generative AI Applications with Firebase Genkit: Challenge Lab

Lab

1 hour

No cost

Intermediate

★★★★☆

Rate Lab

This lab may incorporate AI tools to support your learning.

Challenge Lab Overview

This lab will challenge you to perform actions and automation across products. Instead of following step-by-step instructions, you are given a common business scenario and a set of tasks - you figure out how to complete them on your own! An automated scoring system (shown on this page) provides feedback on whether you have completed your tasks correctly.

When you take a Challenge Lab, you will not be taught Google Cloud concepts. You will need to use your skills to assess how to build the solution to the challenge presented. This lab is only recommended for students who have those skills. Are you up for the challenge?

Lab instructions and tasks

70/100

Challenge Lab Overview

Objective

Setup and requirements

Challenge Scenario

Your challenge

Task 1. Setup your Genkit Project

Task 2. Create a flow with a tool for math

Task 3. Deploy the application to Cloud Run

Congratulations!

Objective

This lab provides an introductory, hands-on experience with Firebase Genkit and the Gemini foundation model. You create a simple application with a flow that leverages prompts, models and tools to ask questions.

You will learn how to:

- Create a new Firebase Genkit project.
- Create flows with Genkit and Gemini.
- Use Genkit's tools to leverage function calling in Gemini.
- Work with the local Genkit tools.
- Deploy the flow to Cloud Run.

Setup and requirements

Before you click the Start Lab button

Read these instructions. Labs are timed and you cannot pause them. The timer, which starts when you click **Start Lab**, shows how long Google Cloud resources will be made available to you.

This Qwiklabs hands-on lab lets you do the lab activities yourself in a real cloud environment, not in a simulation or demo environment. It does so by giving you new, temporary credentials that you use to sign in and access Google Cloud for the duration of the lab.

What you need

To complete this lab, you need:

- Access to a standard internet browser (Chrome browser recommended).
- Time to complete the lab.

Note: If you already have your own personal Google Cloud account or project, do not use it for this lab.

Note: If you are using a Pixelbook, open an Incognito window to run this lab.

How to start your lab and sign in to the Google Cloud console

1. Click the **Start Lab** button. If you need to pay for the lab, a dialog opens for you to select your payment method. On the left is the Lab Details pane with the following:
 - The Open Google Cloud console button
 - Time remaining
 - The temporary credentials that you must use for this lab
 - Other information, if needed, to step through this lab

2. Click **Open Google Cloud console** (or right-click and select **Open Link in Incognito Window** if you are running the Chrome browser).

The lab spins up resources, and then opens another tab that shows the Sign in page.

Tip: Arrange the tabs in separate windows, side-by-side.

Note: If you see the **Choose an account** dialog, click **Use Another Account**.

3. If necessary, copy the **Username** below and paste it into the **Sign in** dialog.

student-82-3fd3c3684cfd@qwiklabs.net

You can also find the Username in the Lab Details pane.

4. Click **Next**.

5. Copy the **Password** below and paste it into the **Welcome** dialog.

AxH1520XK0y0

You can also find the Password in the Lab Details pane.

6. Click **Next**.

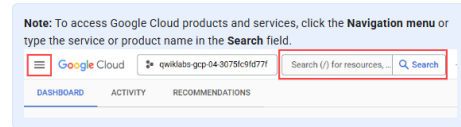
Important: You must use the credentials the lab provides you. Do not use your Google Cloud account credentials.

Note: Using your own Google Cloud account for this lab may incur extra charges.

7. Click through the subsequent pages:

- Accept the terms and conditions.
- Do not add recovery options or two-factor authentication (because this is a temporary account).
- Do not sign up for free trials.

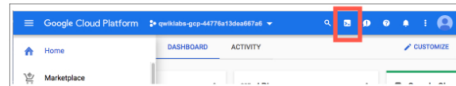
After a few moments, the Google Cloud console opens in this tab.



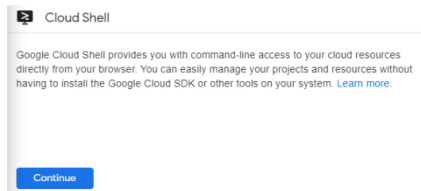
Activate Cloud Shell

Cloud Shell is a virtual machine that is loaded with development tools. It offers a persistent 5GB home directory and runs on the Google Cloud. Cloud Shell provides command-line access to your Google Cloud resources.

In the Cloud Console, in the top right toolbar, click the **Activate Cloud Shell** button.



Click **Continue**.



It takes a few moments to provision and connect to the environment. When you are connected, you are already authenticated, and the project is set to your *PROJECT_ID*. For example:



gcloud is the command-line tool for Google Cloud. It comes pre-installed on Cloud Shell and supports tab-completion.

You can list the active account name with this command:

```
gcloud auth list
```

(Output)

```
Credentialed accounts:
- <myaccount>@<mydomain>.com (active)
```

(Example output)

```
Credentialed accounts:
- google1623327_student@qwiklabs.net
```

You can list the project ID with this command:

```
gcloud config list project
```

(Output)

```
[core]
project = <project_ID>
```

(Example output)

```
[core]
project = qwiklabs-gcp-44776a13dea667a6
```

For full documentation of `gcloud` see the [gcloud command-line tool overview](#).

Challenge Scenario



Here is a company overview, which is provided on Cymbal Shops's website.

Cymbal Shops is a digitally transforming legacy retailer.

It is inspired by clients like: Bed Bath & Beyond, Best Buy, Home Depot, Nordstrom.

Founded in 1974, Cymbal Shops started out as Cymbal Eateries, offering delicious meals crafted by Cymbal Group in Minnesota and neighboring states. As the demand for quality dining experiences grew, the company quickly expanded its offerings to include Cymbal Bites, a quick-service option, and Cymbal Cafés, providing a cozy atmosphere for coffee and pastries.

Today, Cymbal Shops operates 714 locations across North America, encompassing Cymbal Eateries, Bites, Cafés, and other dining concepts. In 2019, the company reported \$15 billion in revenue and employed 80,400 staff members across the United States and Canada.

Your challenge

You work for Cymbal Shops, a chain offering Firebase Genkit. Firebase Genkit is an open source framework that helps you build, deploy, and monitor production-ready AI-powered apps.

Genkit is designed for app developers, to help you easily integrate powerful AI capabilities into your apps with familiar patterns and paradigms. It's built by the same team behind Firebase, leveraging our experience in building tools used by millions of developers worldwide.

In this lab, you will use the Genkit project to leverage function calling from Gemini. You will create a prompt, add a tool to interact with code and encapsulate the process in a flow. You will also test the application through the Genkit Development UI and the CLI interface.

Task 1. Setup your Genkit Project

In this section, you will set up the `Genkit CLI` in **Cloud Shell** and configure it to access the Gemini model deployed in Google Cloud's Vertex AI.

1. In the **Google Cloud Shell**, initialize the environment variables for Google Cloud.

```
export GCP_PROJECT=qwiklabs-gcp-03-9ec807dfba02
export GCP_LOCATION=us-east1
```

2. Authenticate the console to your `project` and `user` using below command.

```
gcloud auth application-default login
```

- When prompted, type **Y**, and press **Enter**.
- Click on the URL provided to open a new browser window that displays a verification code.
- When you're prompted to continue, click **Continue**, and click **Allow**.
- To get the verification code, click **Allow** and click **Copy**.
- Back in the Cloud Shell terminal, where it says `enter the verification code provided in your browser`, paste the copied verification code, and press **Enter**.

3. Run the following command to create a new `node` project. This will create a `package.json` file.

```
mkdir genkit-intro && cd genkit-intro
npm init -y
```

4. Install the `Genkit CLI` by running the following command:

```
npm install -D genkit-cli@1.12.0 tsx typescript
```

5. To install the core Genkit package and other dependencies for your app, run the following commands:

```
npm install genkit@1.12.0 --save
npm install @genkit-ai/vertexai@1.12.0 @genkit-ai/google-cloud@1.12.0 @genkit-ai/express@1.12.0 --save
```

6. Check the `genkit-intro/package.json` file and review all the dependencies that were added. Make sure that the following dependencies are present.

```
"dependencies": {
  "@genkit-ai/express": "^1.12.0",
  "@genkit-ai/google-cloud": "^1.12.0",
  "@genkit-ai/vertexai": "^1.12.0",
  "express": "^4.21.2",
  "genkit": "^1.12.0",
  "zod": "^3.25.62"
}
```

7. Create the application source folder named `src` and main file named `index.ts` in the directory named `genkit-intro`.

```
mkdir src && touch src/index.ts
```

8. Open the `src/index.ts` file and add the below contents to import the required libraries.

```
import { z, genkit } from 'genkit';
import { vertexAI } from '@genkit-ai/vertexai';
import { logger } from 'genkit/logging';
import { startFlowServer } from '@genkit-ai/express';
```

9. Next, import `defineTool` which allows you to add custom tools for your Genkit flows and the `gemini20Flash001` model from Vertex AI. Add these import statements to the top of your `src/index.ts` file.

```
import { defineTool } from '@genkit-ai/ai';
import { gemini20Flash001 } from '@genkit-ai/vertexai';
```

10. Examine the following Genkit framework configuration. Here, you will use the Vertex AI models as plugins, set log level to debug for troubleshooting and implement tracing for monitoring. Add the below code after the import section into `src/index.ts` file.

```
const ai = genkit({
  plugins: [
    vertexAI({ location: 'us-east1' }),
  ],
});
logger.setLogLevel('debug');
enableTracingAndMetrics: true
```

11. Run the below commands in the terminal to move the `package.json` and `index.ts` files to the Cloud Storage Bucket `qwiklabs-gcp-03-9ec807dfba02`.

```
gsutil -m cp -r ~/genkit-intro/package.json gs://qwiklabs-gcp-03-9ec807dfba02
gsutil -m cp -r ~/genkit-intro/src/index.ts gs://qwiklabs-gcp-03-9ec807dfba02
```

Click **Check my progress** to verify the objectives.

☐ Setup your Genkit Project

[Check my progress](#)

Task 2. Create a flow with a tool for math

In this section, we will use an abstraction layer that leverages `function calling` from Gemini to recognize if an external service needs to be called, extract the parameters, identify the function, execute the function and pass the results back to the LLM. Here, we will use this functionality to call two functions to perform **multiplications** and **sums**.

1. Open the `index.ts` file again and define the input and outputs for the tool. Here, you will require an `OperationsSchema` with two parameters `a` and `b` that should be numbers, and a `QuestionSchema` with a parameter to be a string called `question`.

```
// TODO:
export const OperationSchema = ...

// TODO:
export const QuestionSchema = ...
```

2. Define two functions named `multiply` and `sum`. Each function should accept two numerical inputs and return a single numerical output.

```
// TODO:
function multiply(...)

// TODO:
function sum(...
```

3. Define a tool for the `multiply` function and the `sum` function. These tools should accept the `OperationSchema`, should output a number and call the functions defined above.

```
...

```

3

10

10

10

10

```
[Telemetry Server] initialized local file trace store at root: /home
Telemetry API running on http://localhost:4834
Running '/flow/mathOperations' (stream=false)...
Result:
"The answer is 8.\n"
```

```
npx genkit flow:run mathOperations '{"question": "five
times 3"}' | tee -a output.txt
```

Output:

```
[Telemetry Server] initialized local file trace store at root: /home
Telemetry API running on http://localhost:4834
Running '/flow/mathOperations' (stream=false)...
Result:
"The answer is 15.\n"
```

```
npx genkit flow:run mathOperations '{"question": "tell me a
joke"}' | tee -a output.txt
```

Output:

```
[Telemetry Server] initialized local file trace store at root: /home
Telemetry API running on http://localhost:4834
Running '/flow/mathOperations' (stream=false)...
Result:
"Why don't scientists trust atoms? Because they make up everything!\n"
```

10. Run the below commands in the terminal to move the `index.ts` and `output.txt` files to the Cloud Storage Bucket `qwiklabs-gcp-03-9ec807dfba02`.

```
gsutil -m cp -r ~/genkit-intro/src/index.ts gs://qwiklabs-
gcp-03-9ec807dfba02
gsutil -m cp -r ~/genkit-intro/output.txt gs://qwiklabs-
gcp-03-9ec807dfba02
```

Click **Check my progress** to verify the objectives.

☐

Create a flow with a tool for math

☒ Check my progress

Please ensure the content of 'index.ts' and 'output.txt' is as per the instructions.

Task 3. Deploy the application to Cloud Run

In this section, you will use the CLI to deploy your Genkit application to **Cloud Run**, so that anyone can access it over the internet. You can also refer this [tutorial](#).

1. In Cloud Shell, to update `package.json` file, run the following command.

```
npm pkg set main="lib/index.js" scripts.build="tsc -p src"
```

2. Navigate to `genkit-intro/src` folder and create a default Typescript file named `tsconfig.json` with below configurations to build your application.

```
{
  "compilerOptions": {
    "target": "es2016",
    "module": "commonjs",
    "outDir": "../lib",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": ["./**/*"]
}
```

3. Run the following commands:

```
cd ~/genkit-intro
npm install
```

```
npm run build
```

4. Next, deploy the application to **Cloud Run** from code along with the following configurations:

Property	Value
Name	cymbal-genkit-intro
Region	us-east1
Enterprise edition features	Unauthenticated invocations

5. Once the service is deployed, test it by replacing the `service-url` with the URL of the cloud run service named `cymbal-genkit-intro`.

```
curl -X POST {service-url}/mathOperations \
-H "Authorization: Bearer $(gcloud auth print-identity-
token)" \
-H "Content-Type: application/json" -d '{"question": "five
times 3"}'
```

```
    "question": "5 times 5"}}}'
```

Output:

```
{"result": "The answer is 25.\n"}
```

```
curl -X POST {service-url}/mathOperations \
-H "Authorization: Bearer $(gcloud auth print-identity-
token)" \
-H "Content-Type: application/json" -d '{"data": {
"question": "5 plus 5"}}'
```

Output:

```
{"result": "The answer is 10.\n"}
```

Click **Check my progress** to verify the objectives.



Deploy the application to Cloud Run

Check my progress

Assessment completed!

Congratulations!

You have successfully created a Genkit project and leverage function calling from Gemini. Here you created a prompt, added a tool to interact with code and encapsulated the process in a flow. Additionally, you tested the application through the Genkit Development UI and the CLI interface.

Manual Last Updated December 17, 2025

Lab Last Tested December 17, 2025

Copyright 2023 Google LLC All rights reserved. Google and the Google logo are trademarks of Google LLC. All other company and product names may be trademarks of the respective companies with which they are associated.

[< Previous](#)

[Next >](#)